

Парсер новостных сайтов

Техническое описание прототипа v1.0

Дата подготовки: 14.04.2026

1. Общее описание системы

Прототип представляет собой модульную систему автоматического сбора новостей с российских новостных сайтов. Система состоит из трёх независимых компонентов: HTTP-клиента с обходом защиты CloudFlare, XPath-парсера статей и очереди задач на базе RabbitMQ.

Характеристика	Значение
Скорость парсинга	3-10 страниц в минуту
Задержка между запросами	6-12 секунд (случайная)
Обход CloudFlare	Да, через cloudscraper
Хранение задач	RabbitMQ (durable-очереди)
Конфигурация сайтов	YAML-файл с XPath-селекторами
Кодировка	UTF-8 (полная поддержка кириллицы)
Отказоустойчивость	Retry x3, backoff, NACK
Протестировано на сайтах	lenta.ru, rbc.ru, tass.ru

2. Архитектура системы

Система построена по принципу producer-consumer с разделением компонентов сбора и обработки данных.

Scraper	src/scraper.py	HTTP-клиент: cloudscraper + pika, ротация UA, rate limiting
Parser	src/parser.py	XPath-парсинг заголовка, текста, автора, даты

Models	src/models.py	Dataclass Article и ParseTask, сериализация JSON
Queue	src/queue.py	Producer/Consumer над pika, переподключение, DLQ
Producer CLI	scripts/producer.py	Добавление URL в очередь из командной строки
Consumer CLI	scripts/consumer.py	Воркер: читает задачи, парсит, публикует результат
Конфиг	config/sites.yaml	XPath-селекторы для каждого сайта

3. Поток обработки данных

- Шаг 1. Оператор добавляет URL статьи в RabbitMQ через producer.py или HTTP-вызов
- Шаг 2. Consumer читает задачу из очереди news_urls (prefetch=1)
- Шаг 3. Scraper загружает страницу: если сайт использует CloudFlare — используется cloudscraper, иначе requests
- Шаг 4. Соблюдается rate limit: случайная пауза 6-12 секунд между запросами
- Шаг 5. Parser применяет XPath-селекторы из YAML конфига: title, text, author, date
- Шаг 6. Статья валидируется: наличие заголовка и текста от 100 символов
- Шаг 7. Валидная статья публикуется в очередь news_articles, ошибка — в news_errors
- Шаг 8. Consumer подтверждает обработку (ACK) или отправляет в DLQ (NACK без requeue)

Схема очередей RabbitMQ:

news_urls	JSON: ParseTask (url, site_key, retry_count)	24 часа
news_articles	JSON: Article (все поля статьи)	24 часа
news_errors	JSON: url, site_key, error, status	24 часа

4. Конфигурация парсинга сайтов

Каждый сайт описывается в файле `config/sites.yaml`. Для каждого поля (`title`, `text`, `author`, `date`) указывается список XPath-селекторов в порядке приоритета. Первый ненулевой результат используется как значение поля.

Сайт	CloudFlare	Примечания по структуре
lenta.ru	Да	Заголовок: <code>topic-body__title</code> . Текст: <code>topic-body__content//p</code> . Автор: <code>topic-authors__link</code> . Дата: <code>time/@datetime</code>
rbc.ru	Да	Заголовок: <code>article__header__title</code> . Текст: <code>article__text//p</code> . Автор: <code>article__authors__author_name</code> . Дата: <code>@content</code> атрибут <code>span</code>
tass.ru	Нет	Заголовок: <code>h1[contains(@class,'title')]</code> . Текст: <code>TextContent//p</code> . Автор: <code>span[contains(@class,'author')]</code> . Дата: <code>time/@datetime</code>

Добавление нового сайта не требует изменения кода. Достаточно добавить новую секцию в YAML-файл с XPath-селекторами и перезапустить `consumer`.

5. Надёжность и отказоустойчивость

Retry с backoff. При ошибках HTTP 403/429/503 система делает до 3 повторных попыток с экспоненциальной задержкой ($2^n + \text{jitter}$ секунд).

NACK без `requeue`. При критической ошибке обработки задача отправляется в очередь ошибок `news_errors`, а не возвращается в основную очередь.

Durable очереди. Все очереди помечены `durable=True`. Сообщения сохраняются на диск и переживают перезапуск RabbitMQ.

Persistent messages. Каждое сообщение публикуется с `delivery_mode=Persistent`.

Переподключение. Консьюмер автоматически переподключается к RabbitMQ при разрыве соединения.

Prefetch=1. Воркер берёт только одну задачу за раз, что предотвращает перегрузку и потерю сообщений.

Валидация данных. Статья принимается только если есть заголовок и текст от 100 символов.

6. Инструкция по запуску

Шаг 1. Установка зависимостей:

```
pip install -r requirements.txt
```

Шаг 2. Запуск RabbitMQ через Docker:

```
docker-compose up -d
```

Панель управления RabbitMQ будет доступна по адресу <http://localhost:15672> (логин: guest, пароль: guest).

Шаг 3. Настройка переменных окружения:

```
cp .env.example .env # при необходимости отредактируйте
```

Шаг 4. Тест без очереди (быстрая проверка):

```
python scripts/test_parser.py
```

Шаг 5. Запуск консьюмера (воркер):

```
python scripts/consumer.py
```

Шаг 6. Добавление URL в очередь:

```
python scripts/producer.py --demo python scripts/producer.py --site lenta_ru --url  
https://lenta.ru/news/... python scripts/producer.py --status
```

7. Возможности расширения

- Добавление новых сайтов без изменения кода (только YAML)
- Горизонтальное масштабирование: несколько экземпляров consumer на один брокер
- Интеграция с базой данных: PostgreSQL, MongoDB, Elasticsearch
- Экспорт в Google Sheets или S3
- Дедупликация статей по URL или хешу текста
- Планировщик: автоматическая подача URL через cron или APScheduler
- Мониторинг через RabbitMQ Management API или Prometheus
- Telegram-уведомления о ходе парсинга
- Поддержка JavaScript-сайтов через Playwright/Selenium

8. Технический стек

cloudscraper	1.2.71	Обход CloudFlare
lxml	5.2.2	XPath-парсинг HTML
pika	1.3.2	RabbitMQ AMQP клиент
PyYAML	6.0.2	Конфигурация сайтов
requests	2.32.3	HTTP-запросы
python-dateutil	2.9.0	Нормализация дат
reportlab	4.2.2	Генерация PDF-отчётов
python-dotenv	1.0.1	Управление env-переменными

RabbitMQ	3.13	Брокер сообщений (Docker)
Python	3.10+	Язык реализации

Техническое описание подготовлено автоматически. Версия 1.0. Дата: 14.04.2026
Система разработана как масштабируемый production-ready прототип.